FAIL-SAFE FIRMWARE UPDATE SYSTEM / OTA

SanDisk Hackathon | Track: Firmware
Theme: Fail-Safe Firmware Update System / OTA

Team: Three Body Problem

Platform	ESP32 (ESP-IDF + FreeRTOS)
Language	C (firmware) | Python (backend) | Rust (CLI tool)
Protocol	MQTT over WiFi | TLS 1.3 | QoS Level 1
Core Feature	A/B Dual Partition + Native ESP32 Bootloader Rollback
Safety Model	3-Stage Commit: Download → Verify → Trial Boot → Commit
Backend	FastAPI + Mosquitto Broker + Streamlit Dashboard

1. Problem Statement

Firmware updates are a critical maintenance operation for embedded IoT devices deployed in the field. However, a failed or interrupted update — caused by power loss, network disruption, hardware fault, or corrupted binaries — can permanently brick a device, making it non-recoverable without physical intervention. This is especially severe in large-scale IoT deployments where physical access is impractical.

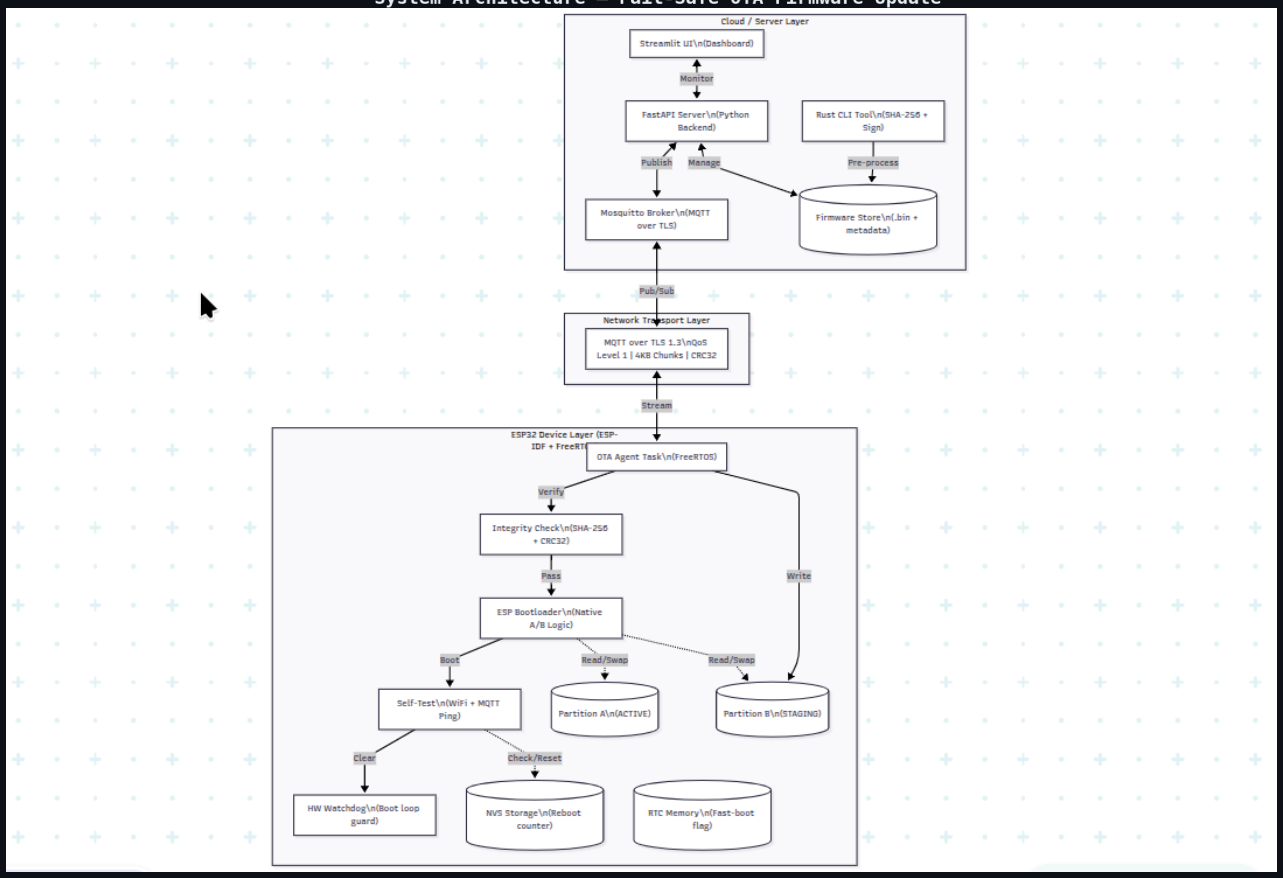
The challenge is to design a **Fail-Safe Over-The-Air (OTA) Firmware Update System** for an ESP32 microcontroller that guarantees the device always remains in a known-good, operational state regardless of what goes wrong during the update process. The system must handle partial downloads, corrupted binaries, post-update boot failures, and network instability — all without human intervention.

Challenge	Risk	Our Solution
Power loss mid-flash	Corrupted staging partition	Atomic write to Partition B only
Corrupted binary download	Boot into broken firmware	SHA-256 + CRC32 verification
New firmware boot failure	Device becomes unresponsive	ESP32 PENDING_VERIFY + auto-rollback
Boot loop / repeated crash	Infinite reboot cycle	NVS reboot counter, rollback at 3
Network disconnect mid-transfer	Incomplete firmware on device	Chunked MQTT with resume support

2. Detailed Architecture

The system is organized into three layers: the Cloud/Server Layer, the Network Transport Layer, and the ESP32 Device Layer. Each layer has a clear responsibility, and failure in any one layer is handled gracefully without affecting the others.

System Architecture — Fail-Safe OTA Firmware Update



2.1 Layer Breakdown

- **Cloud / Server Layer:** FastAPI Python server manages firmware versions, stores .bin files, and triggers updates via MQTT publish. A Rust CLI tool pre-processes each firmware binary to generate SHA-256 hash and metadata JSON before publishing.
- **Network Layer:** MQTT over TLS 1.3 via a self-hosted Mosquitto broker. QoS Level 1 guarantees at-least-once delivery. Firmware is split into 4KB chunks — each chunk carries a sequence number and CRC32 checksum.
- **ESP32 Device Layer:** An OTA Agent FreeRTOS task handles download, verification, and the write to Partition B. The ESP32 native bootloader manages the A/B partition swap and rollback logic at the hardware level.

3. Detailed Methodology

3.1 A/B Dual Partition Strategy

The ESP32 flash is divided into two equal firmware partitions — Partition A (active) and Partition B (staging). The device always boots from the partition marked active in the OTA Data Partition. New firmware is written exclusively to the staging partition, ensuring the running firmware is never touched during an update. This is the foundational safety guarantee.

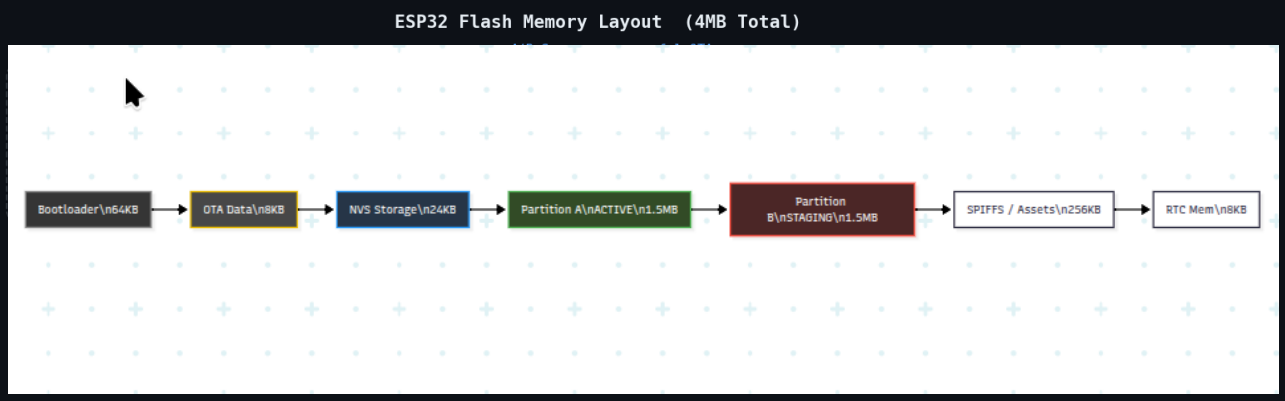


Figure 2 — ESP32 Flash Memory Layout with A/B Partitioning

3.2 The 3-Stage Commit Protocol

The update lifecycle follows a strict 3-stage protocol, leveraging the ESP32 bootloader's native state machine. This is not custom rollback logic — it is hardware-guaranteed.

Stage 1 — Download & Verify

The OTA Agent subscribes to MQTT firmware topics. Chunks arrive sequentially; each is validated with CRC32 before being written to Partition B. After all chunks, a full SHA-256 hash of the written binary is compared against the server-published hash.

Stage 2 — Trial Boot (PENDING_VERIFY)

On rebooting into the new firmware, the ESP32 bootloader marks it as ESP_OTA_IMG_PENDING_VERIFY. The device is now in a trial state. If it crashes or hangs at any point before the commit call, the bootloader automatically aborts and reverts to Partition A on the next reboot.

Stage 3 — Self-Test & Commit

The new firmware runs a self-test: verify WiFi reconnects, ping the MQTT broker, and confirm the firmware version in NVS matches the expected version. On all-pass, call esp_ota_mark_app_valid_cancel_rollback() to permanently commit. On any failure — do nothing. The bootloader handles the rest.

3.3 Complete OTA State Flow

The flowchart below captures every decision point in the update process, including all failure paths and the automatic rollback triggered by the bootloader.

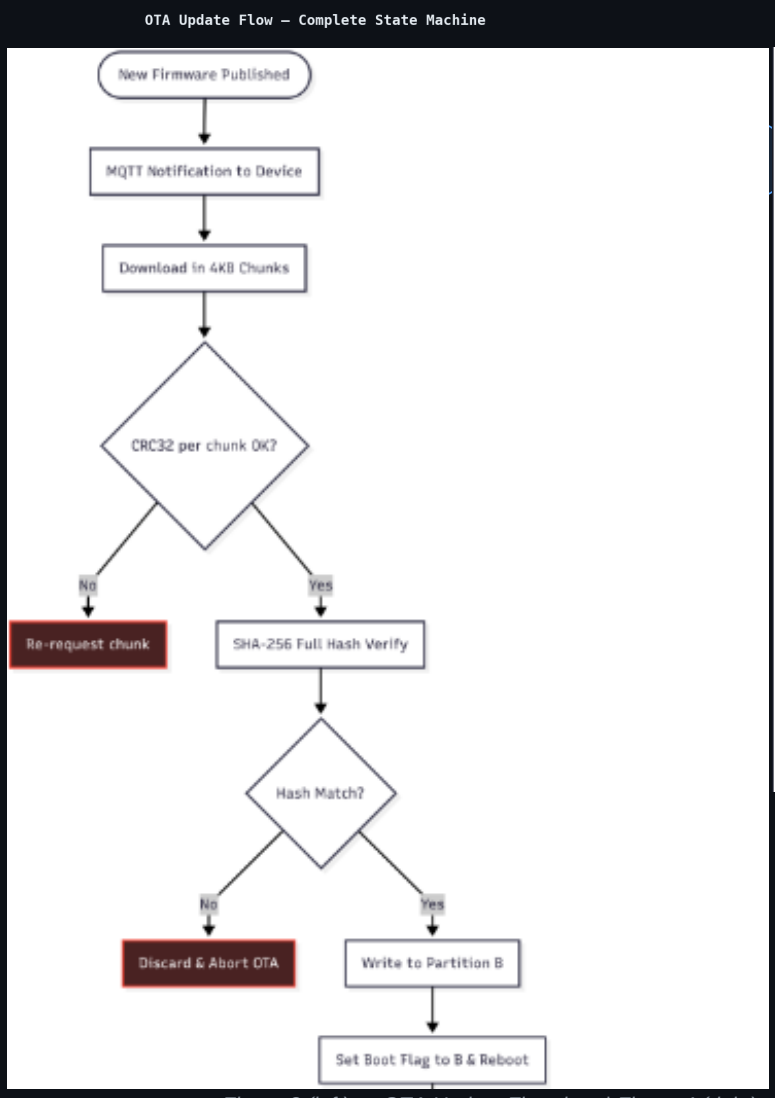


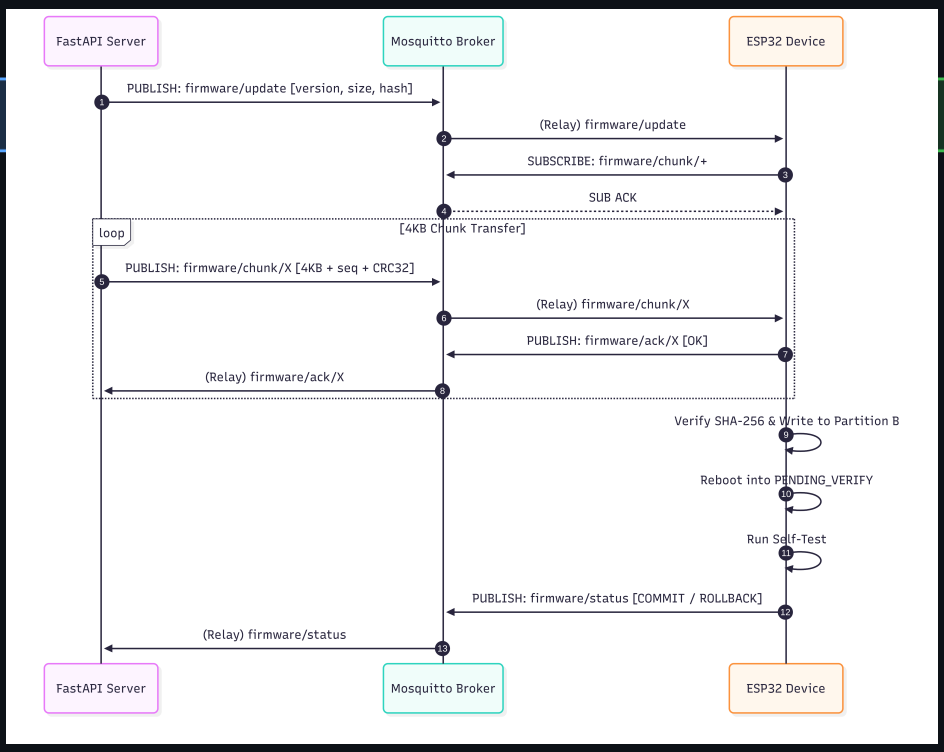
Figure 3 (left) — OTA Update Flowchart | Figure 4 (right) — Boot Loop Detection State Machine

3.4 Boot Loop Detection

On every boot, the firmware increments a reboot counter stored in NVS. After a successful self-test and commit, the counter is reset to 0. If the counter reaches 3 without a commit, the bootloader automatically rolls back to the previously committed partition. This handles scenarios where the device crashes during the self-test window before the counter can be reset.

4. MQTT Communication Protocol

MQTT is chosen for its lightweight pub/sub model, ideal for constrained embedded devices. The broker (Mosquitto) acts as the decoupling layer between the backend server and the device fleet. TLS 1.3 secures all communication. QoS Level 1 ensures firmware chunks are delivered at least once, with the device-side deduplication using sequence numbers.



4.1 Topic Structure

Topic	Publisher	Subscriber	Purpose
`firmware/update`	Server	ESP32	Announce new version + hash + size
`firmware/chunk/{seq}`	Server	ESP32	Deliver 4KB chunk with CRC32
`firmware/ack/{seq}`	ESP32	Server	Acknowledge chunk receipt
`firmware/status`	ESP32	Server	Report VERIFY_OK / COMMIT / ROLLBACK
`device/{id}/status`	ESP32	Dashboard	Heartbeat + current firmware version

5. Technology Stack

Component	Technology	Justification
MCU Platform	ESP32 (ESP-IDF)	Built-in WiFi, native OTA partition support
RTOS	FreeRTOS	Bundled with ESP-IDF, task-based OTA agent
Firmware Language	C	Native to ESP-IDF, zero toolchain friction
OTA Mechanism	esp_ota_ops.h API	Hardware-guaranteed A/B partition switching
Communication	MQTT + TLS 1.3	Lightweight, pub/sub, QoS delivery guarantees
MQTT Broker	Mosquitto (self-hosted)	Simple setup, no cloud dependency
Backend	Python + FastAPI	Rapid dev, clean REST API for firmware mgmt
Hash / Signing Tool	Rust CLI (Axum-free)	SHA-256 + metadata gen, safe and fast
Dashboard	Streamlit	Quick visual — device version + update status
Verification	SHA-256 + CRC32	Full binary + per-chunk integrity checks
Persistence	NVS + RTC Memory	Reboot counter + fast boot flag storage

6. Expected Deliverables

D1	**ESP32 OTA Firmware (C / ESP-IDF)** Complete firmware with FreeRTOS OTA Agent task, MQTT subscriber, chunk handler, CRC32 validator, SHA-256 verifier, NVS reboot counter, self-test routine, and esp_ota_mark_app_valid_cancel_rollback() commit call.
D2	**Python FastAPI Backend** REST API to upload and manage firmware versions. Stores binaries and metadata. Triggers MQTT publish to Mosquitto broker on new release.
D3	**Rust CLI Firmware Tool** Command-line tool that takes a .bin firmware file, computes SHA-256 hash, generates version metadata JSON, and outputs a signed package ready for the server.
D4	**Mosquitto MQTT Broker Config** Configured broker with TLS certificates, topic ACLs, and QoS settings. Docker-compose file for one-command deployment.
D5	**Streamlit Dashboard** Real-time web UI showing connected devices, their current firmware version, last update timestamp, and update status (PENDING / COMMITTED / ROLLED BACK).
D6	**partition_table.csv** Custom ESP32 partition table defining Bootloader, OTA Data, NVS, Partition A, Partition B, and SPIFFS regions with correct sizes and offsets.
D7	**Demo Video + Working Prototype** Live demo showing: normal OTA update commit flow, and deliberately broken firmware triggering automatic rollback — the device recovers without human intervention.
D8	**Technical Documentation** This report + inline code documentation + API reference + setup/run guide.

7. SanDisk / Flash Memory Relevance

This project directly addresses core challenges in flash memory management — central to SanDisk's domain of expertise.

■ Write Endurance Awareness

OTA writes are among the heaviest flash operations. Our A/B design concentrates writes to Partition B during updates, protecting Partition A (active) from unnecessary wear cycles.

■ Atomic Write Guarantees

Flash memory does not support true atomic multi-block writes. Our system writes only to the staging partition and only commits by updating a single OTA Data flag — mimicking atomic semantics over NOR flash.

■ NVS as Reliable Metadata Store

The NVS (Non-Volatile Storage) library implements its own wear leveling over the flash, making it safe to store frequently-updated counters like the reboot counter.

■ NAND vs NOR Considerations

ESP32 uses NOR flash — byte-addressable and suitable for XIP (execute-in-place). Our design acknowledges this: Partition A runs XIP while Partition B is written page-by-page.

■ Power-Loss Resilience

Unlike storage devices that buffer writes, embedded NOR flash writes are immediate but non-atomic across pages. Our CRC32 per-chunk and full SHA-256 detect any partial page corruption from power loss during download.

8. Future Scope

Feature	Description
Delta / Differential OTA	Use bsdiff to generate patch files — reducing OTA payload by 60-80%
ECDSA-256 Code Signing	Sign firmware at build time; device verifies signature before flash
Fleet Management	Server tracks 1000s of devices, staged rollouts, canary deployments
LTE Fallback	SIM800 module as backup when WiFi unavailable for remote deployments
Secure Boot	ESP32 eFuse-based secure boot chain from ROM to application
Cloud Migration	Move from local Mosquitto to AWS IoT Core for production scale

The system guarantees: a device running this OTA stack can never be permanently bricked by a failed update. The bootloader always has a path back.